Appium 上的 iOS测试

Weibo: 晋恒温

TesterHome 出品

testerhome.com

QQ群: 315508626

TesterHome
社区
Wiki
Blog
酷站
会员
搜索
0
lihuazhang
Home / 首页
Feed
国内首个移动测试交流社区，专注于Web测试，移动测试，测试产品与云测服务，一个技术驱动的组织，一个活跃而温馨的测试大家庭。
欢迎关注微信账号：测试圈 官方QQ群：315508626
社区精华贴
论坛资源整理贴
自动化测试工具 • lihuazhang • 于5小时前发布
TesterHome 第一次公开课资料整理
Appium • seveniruby • 最后由 rockchensir 于1天前回复
1
[翻译] Android and iOS Support 来自 Official Se...
资讯点评 • lihuazhang • 最后由 seveniruby 于4天前回复
1
Appium 故障调试 Troubleshooting Appium 中文...
Appium • lihuazhang • 最后由 seveniruby 于5天前回复
1

TesterHome

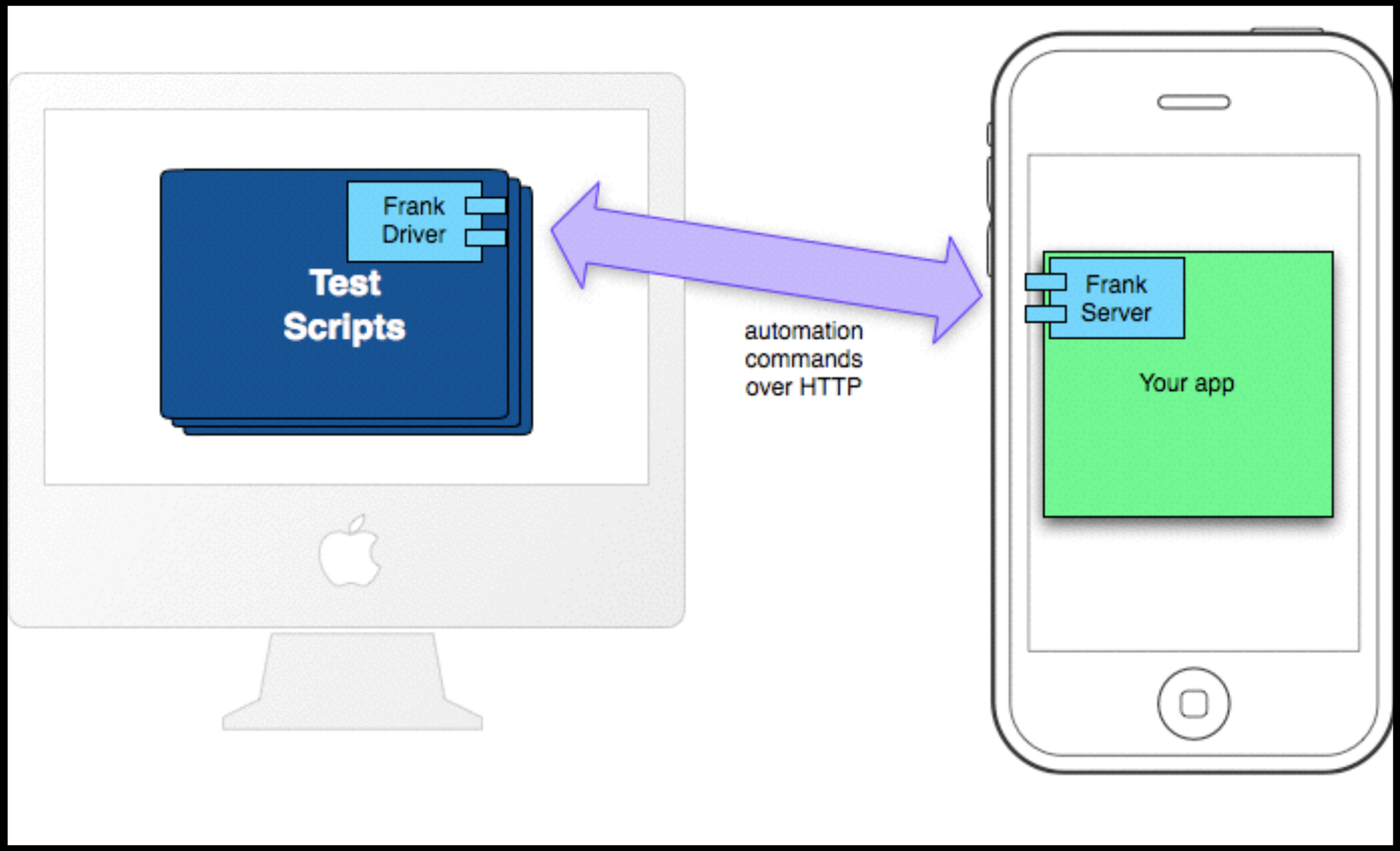
公开课

- 高手在民间
- 人人都能做老师
- 分享就是得到

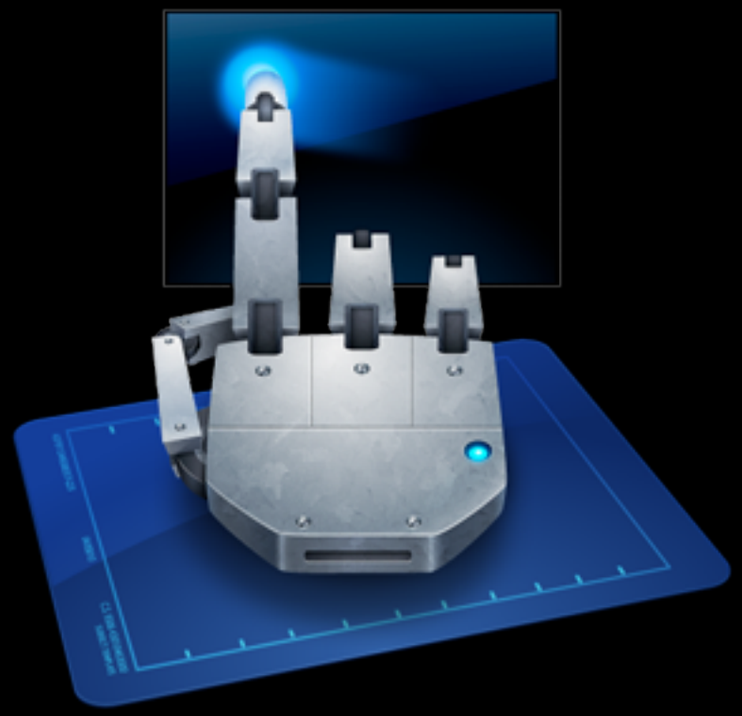
iOS 测试工具

工具	语言	测试框架	支持其他移动平台	方法	把相同的app部署至应用商店	支持测试web-views
Frank	Ruby	Cucumber	Also supports OSX apps	Requires you to embed a symbiote server in your app and uses undocumented APIs	NO	NO (only via JavaScript calls)
KIF	Objective C	OCUnit/SenTest	NO	Modifies your app to use undocumented APIs	NO	NO
Subliminal	Objective C	OCUnit/SenTest	NO	Embeds into your project but uses UIAutomation instead of undocumented APIs	NO	NO
Zucchini	Custom DSL (CoffeeScript/Ruby)	Custom DSL	NO	Generates UIAutomation JavaScript that is executed against your app	YES	NO
Calabash	Ruby	Cucumber	Also supports Android Apps	Requires you to embed a server in your app to control it	NO	YES
Appium	Ruby, C#, Java, JavaScript, Objective C, PHP, Python, Perl, Clojure	Agnostic	Also supports Mac OSX apps, Android Firefox OS	Uses Instruments to control app using the standard WebDriver WIRE protocol	YES	YES
ios-driver	Ruby, C#, Java, JavaScript, Objective C, PHP, Python, Perl, Clojure	Agnostic	NO	Uses Instruments to control app using the standard WebDriver WIRE protocol	YES	YES

Calaba.sh


Frank
Driver
Test
Scripts
automation
commands
over HTTP
Frank
Server
Your app

侵入型



基于 UIAutomation 型

Rules

- Test the same app you submit to the marketplace
- Write your tests in any language, using any framework
- Use a standard automation specification and API
- Build a large and thriving open- source community effort

我们选择
UI Automation

Instruments

- Instruments是用于动态调追踪和分析OS X和iOS的代码的性能分析和测试工具。
- Instruments支持多线程的调试。
- 可以用Instruments去录制和回放，图形用户界面的操作过程。
- 可将录制的图形界面操作和Instruments保存为模板，供以后访问使用。
- Xcode 3.0+和OS X v10.5+ 才有Instruments。

可以使用Instruments干的事情包括:

- 追踪代码中的（甚至是那些难以复制的）问题
- 分析程序的性能
- 实现程序的自动化测试
- 部分实现程序的压力测试
- 执行系统级别的通用问题追踪调试
- 是你对程序的内部运行过程更加了解

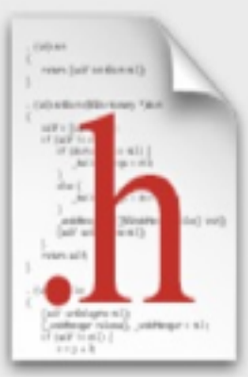
UIAutomation

- UIAutomation 是 instruments 中的一个实现 UI 自动化的框架。
- iOS4 之后才有的。

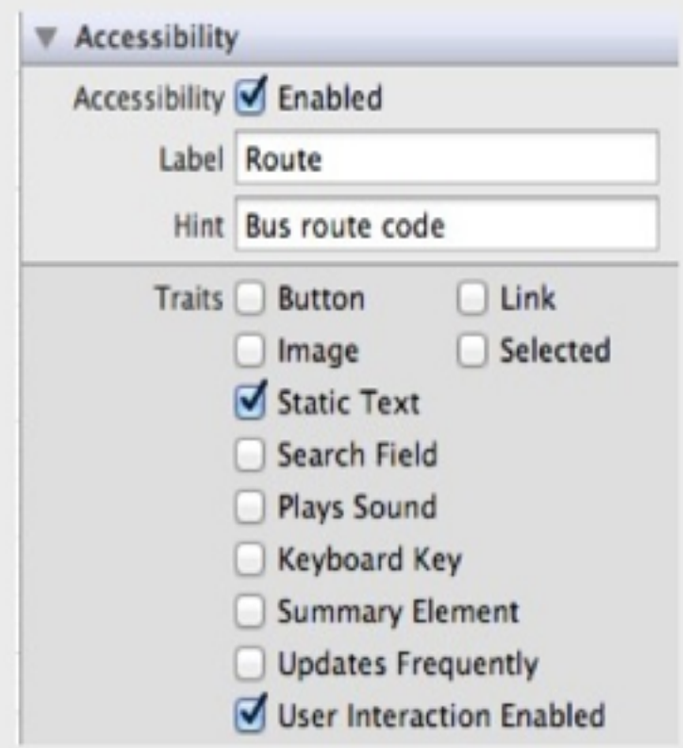
```
var target = UIATarget.localTarget();
var app = target.frontMostApp();
var window = app.mainWindow();
var tableView = window.tableViews()[0];
var cells = tableView.cells();

UIALogger.logMessage("Turn on edit mode");
var navigationBar = window.navigationBar();
var editButton = navigationBar.leftButton();
editButton.tap();

UIALogger.logMessage("Delete cell named 'coffee'");
var coffeeCell = cells["coffee"];

var deleteSwitch = coffeeCell.switches()[0];
deleteSwitch.tap();

var deleteButton = coffeeCell.buttons()[0];
deleteButton.tap();

target.pushTimeout(0.1);
coffeeCell.waitForInvalid();
target.popTimeout();

if (coffeeCell.isValid()) {
    UIALogger.logError("Oops. Cell is still there!");
    throw new Error("Oops. Cell is still there!");
} else {
    UIALogger.logMessage("Cell is gone, we're all set!");
}
```

- UI Automation 使用 JavaScript 语法，苹果提供 UI Automation JavaScript Reference
- App UI 以 Element hierarchy （元素层级） 展示。
- 可以在UI 结构中按 Position 定位元素，也可以设置 UI Element Accessibility，通过 label 定位元素

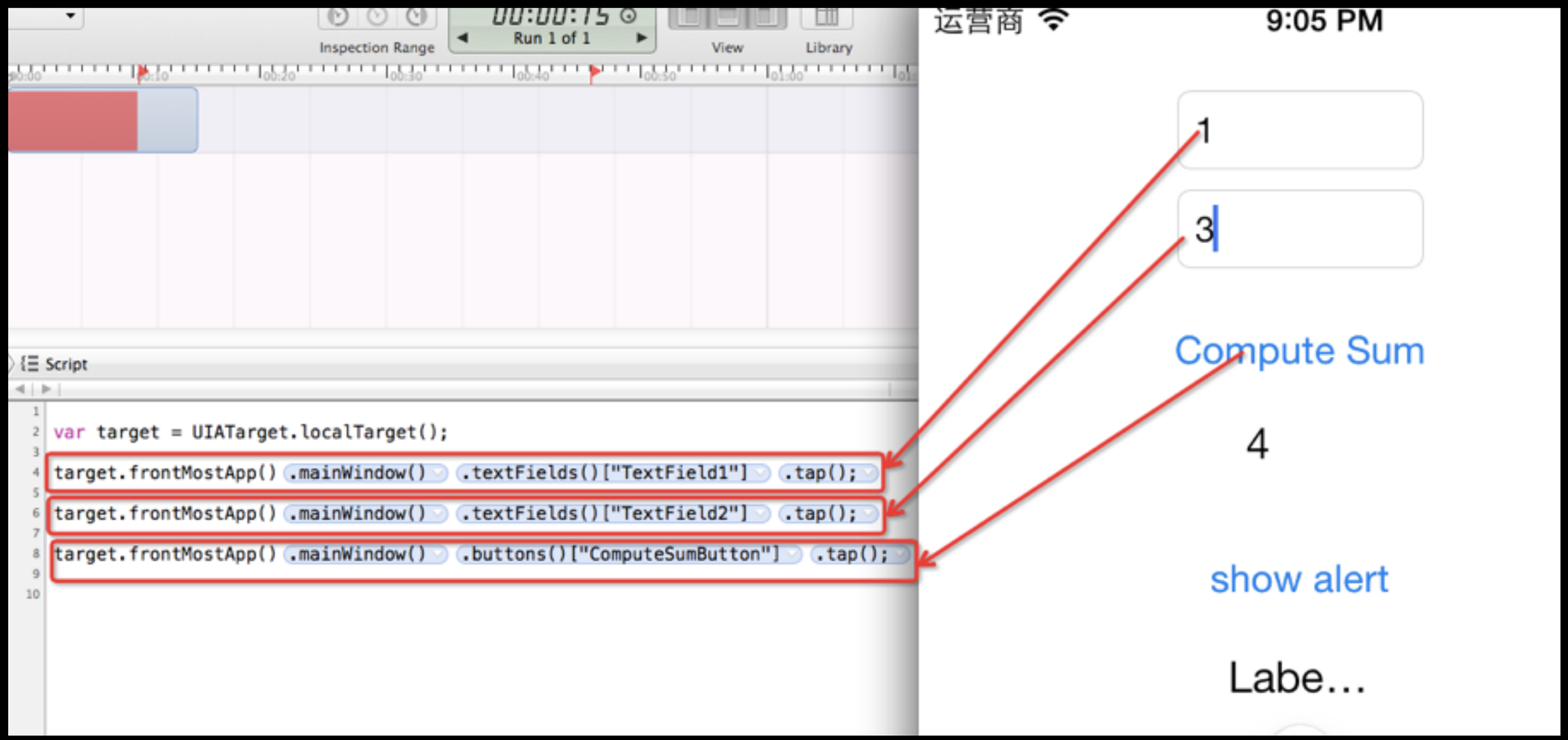
```
self.navigationItem.leftBarButtonItem = [[UIBarButtonItem alloc] initWithImage:[UIImage imageNamed:@"lbs_arrow"]
                                                                         style:UIBarButtonItemStyleBordered
                                                                        target:self
                                                                        action:@selector(onCrossHairsButton:)];
self.navigationItem.leftBarButtonItem.accessibilityLabel =
    NSLocalizedString(@"Locate Me", @"");
self.navigationItem.leftBarButtonItem.accessibilityHint =
    NSLocalizedString(@"Finds your current location and reloads the map with nearby stops.", @"");
```



UIAccessibility.h



UI Automation 可以录制但是录制出来的脚本可用性并不高。



代码示例

```
var target = UIATarget.localTarget();
var app = target.frontMostApp();
var window = app.mainWindow();

window.textFields()['TextField1'].setValue(4);
window.textFields()['TextField2'].setValue(2);
window.buttons()['ComputeSumButton'].tap();

window.logElementTree();

UIALogger.logMessage(window.staticTexts()[0].value());

if (window.staticTexts()[0].value() == 6) {
   UIALogger.logPass("Test pass!");
} else {
   UIALogger.logFail("Test Failed!");
}
```

更多的 UIAutomation

简单轻量级的框架

- Tuneup JS
- Jasmine

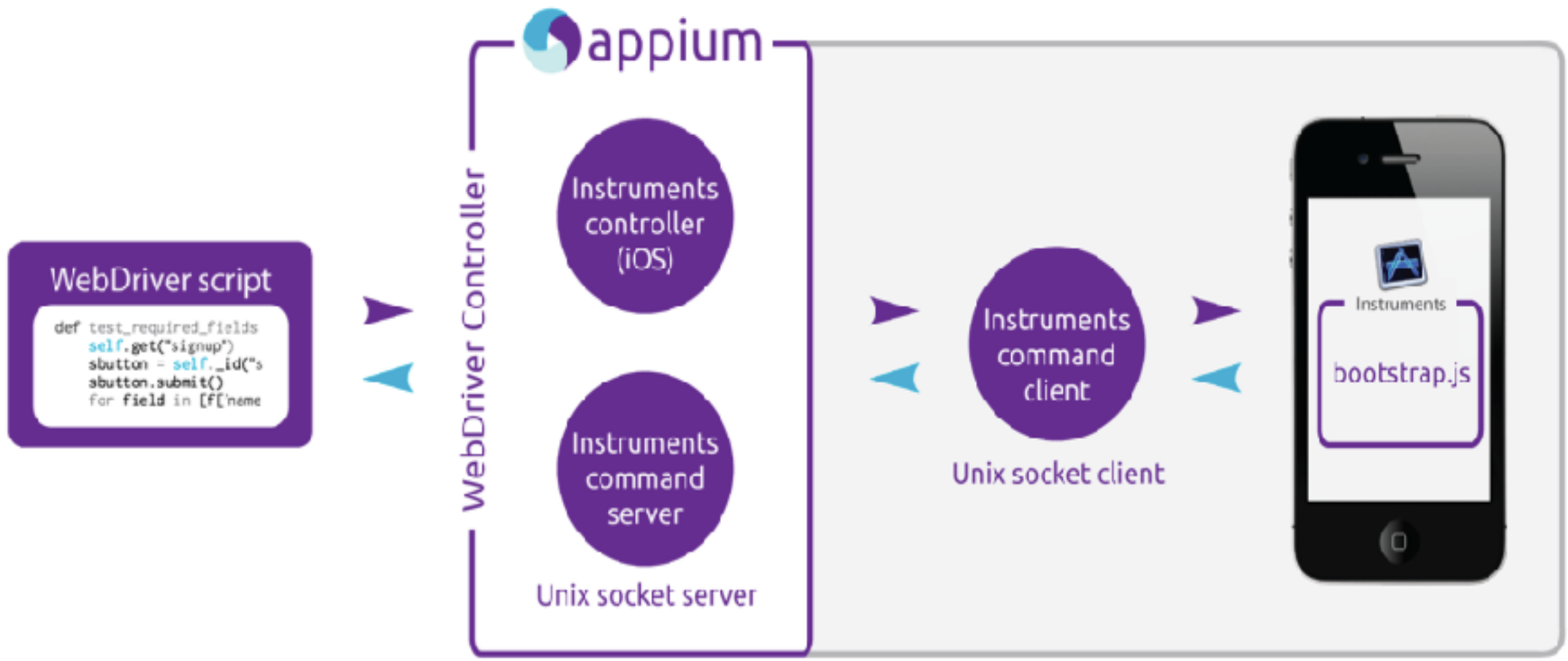
书籍推荐

- Test iOS Apps with UI Automation

但是...

- 只能在 Instruments.app 调试代码
- xCode 不是一般难用
- 只能用 Javascript 写测试用例
- Javascript 不是一般的难调试
- 不能和现有的 Selenium 自动化集成

Appium


appium
WebDriver script
def test_required_fields
self.get("signup")
sbutton = self._id("s
sbutton.submit()
for field in [f['name
WebDriver Controller
Instruments controller (iOS)
Instruments command server
Unix socket server
Instruments command client
Unix socket client
Instruments
bootstrap.js

appium 是一个 HTTP 服务器
创建和处理 WebDriver 的 session

Server 启动过程

- grunt-helpers.js
- app
- port

那这个 server 是神马?

```
module.exports.startAppium = function(appName
  var app;
  if (appName) {
    app = (fs.existsSync(appName)) ? appName:
      path.resolve(__dirname, "sample-code",
  } else {
    app = null;
  }
  return server.run({
    app: app
    , udid: null
    , quiet: !verbose
    , port: 4723
    , nativeInstrumentsLib: false
    , fullReset: false
    , noReset: false
    , launch: app ? true : false
    , log: path.resolve(__dirname, "appium.lo
    , address: '127.0.0.1'
    , androidDeviceReadyTimeout: 5
    , nodeconfig: null
    , robotPort: -1
    , robotAddresss: '0.0.0.0'
    , keepArtifacts: false
    , ipa: null
    , avd: null }
    , readyCb
    , doneCb
  );
```

main.js

- var server = require('./lib/server/main.js')
- 创建 http server
- 配置路由（在实例化 appium 中完成）
- 使用默认值实例化 appium
- 开始监听

```
var main = function(args, readyCb, doneCb) {

  if (args.showConfig) {…
  }

  checkArgs(args);
  if (typeof doneCb === "undefined") {
    doneCb = function() {};
  }
  var rest = express()
    , server = http.createServer(rest);

  rest.configure(function() {…
  });

  // Instantiate the appium instance
  var appiumServer = appium(args);

  // Hook up REST http interface
  appiumServer.attachTo(rest);

  async.series([…
  ], function(err) {…
  });

  server.on('close', doneCb);
  return server;
};
```

appium.js

- appium = require('../appium.js')
- 定义了一系列的方法，当请求来的时候供调用，比如：
  - 判断设备类型
  - 配置待测应用
  - 初始化设备

```
var Appium = function(args) {
  this.args = args;
  this.serverArgs = _.clone(args);
  this.rest = null;
  this.webSocket = null;
  this.deviceType = null;
  this.device = null;
  this.sessionId = null;
  this.desiredCapabilities = {};
  this.oldDesiredCapabilities = {};
  this.session = null;
  this.tempFiles = [];
  this.preLaunched = false;
  this.fastReset = !this.args.fullReset &&
  this.sessionOverride = this.args.sessior
  this.resetting = false;
  this.defCommandTimeoutMs = 60 * 1000;
  this.commandTimeoutMs = this.defCommandT
  this.origCommandTimeoutMs = this.commanc
  this.commandTimeout = null;
  this.isSafariLauncherApp = false;
  this.origAppPath = null;
};
```

第一个请求来的时候...

```
self.driver = webdriver.Remote(
command_executor='http://127.0.0.1:4723/wd/hub',
desired_capabilities={
 'browserName': '',
 'device': 'iPhone Simulator',
 'platform': 'Mac',
 'version': '6.0',
 'app': app
})
```

webdriver 在初始化的时候就会和 server 建立连接，并创建session。

创建 session

- 将代码里的desiredCapabilities 参数传给 appium 的 start 函数。（appium.js）
- start 函数使用 desiredCapabilities 传过来的参数初始化设备类型和其他信息。
- 之后在调用 invoke 函数，invoke 里初始化了设备 this.initDevice() =》 this.device = new IOS(iosOpts)
- 最后 invoke 函数里启动设备 this.device.start(onStart, onDie);

```
exports.createSession = function(req, res) {
  if (typeof req.body === 'string') {
    req.body = JSON.parse(req.body);
  }

  var desired = req.body.desiredCapabilities;

  var next = function(reqHost, sessionId) {
    var redirect = function() {
      res.set('Location', "http://"+reqHost+"/wd/hub/session/" + ses
      res.send(303);
    };
    if (desired && _.has(desired, 'newCommandTimeout')) {
      req.appium.setCommandTimeout(desired.newCommandTimeout, redire
    } else {
      redirect();
    }
  };
  if (req.appium.preLaunched && req.appium.sessionId) {
    req.appium.preLaunched = false;
    next(req.headers.host, req.appium.sessionId, req.appium.device,
  } else {
    req.appium.start(req.body.desiredCapabilities, function(err, ins
      if (err) {
        logger.error("Failed to start an Appium session, err was: "
        respondError(req, res, status.codes.SessionNotCreatedExcepti
      } else {
        logger.info("Appium session started with sessionId " + req.a
        next(req.headers.host, req.appium.sessionId, instance);
      }
    });
  }
```

iOS设备的启动过程

iOS.js

- this.preCleanup.bind(this)
- this.setXcodeFolder.bind(this)
- this.setXcodeVersion.bind(this)
- this.setiOSSDKVersion.bind(this)
- this.detectTraceTemplate.bind(this)
- this.detectUdid.bind(this)
- this.parseLocalizableStrings.bind(this)
- this.setLocale.bind(this)
- this.startLogCapture.bind(this)
- this.setDeviceAndLaunchSimulator.bind(this)
- this.installToRealDevice.bind(this)
- **this.startInstruments.bind(this)**
- this.onInstrumentsLaunch.bind(this),
- this.setBundleId.bind(this),
- this.setInitialOrientation.bind(this),

StartInstruments

- appium 封装了 instruments —— instruments.js
- startInstruments 初始化instruments，并启动 instruments
- instruments 启动的时候，启动了一个 socket server，默认监听在 /tmp/instruments_sock 同时分了一个线程出来运行真正的 instruments.app，将 bootstrap.js 打入设备。

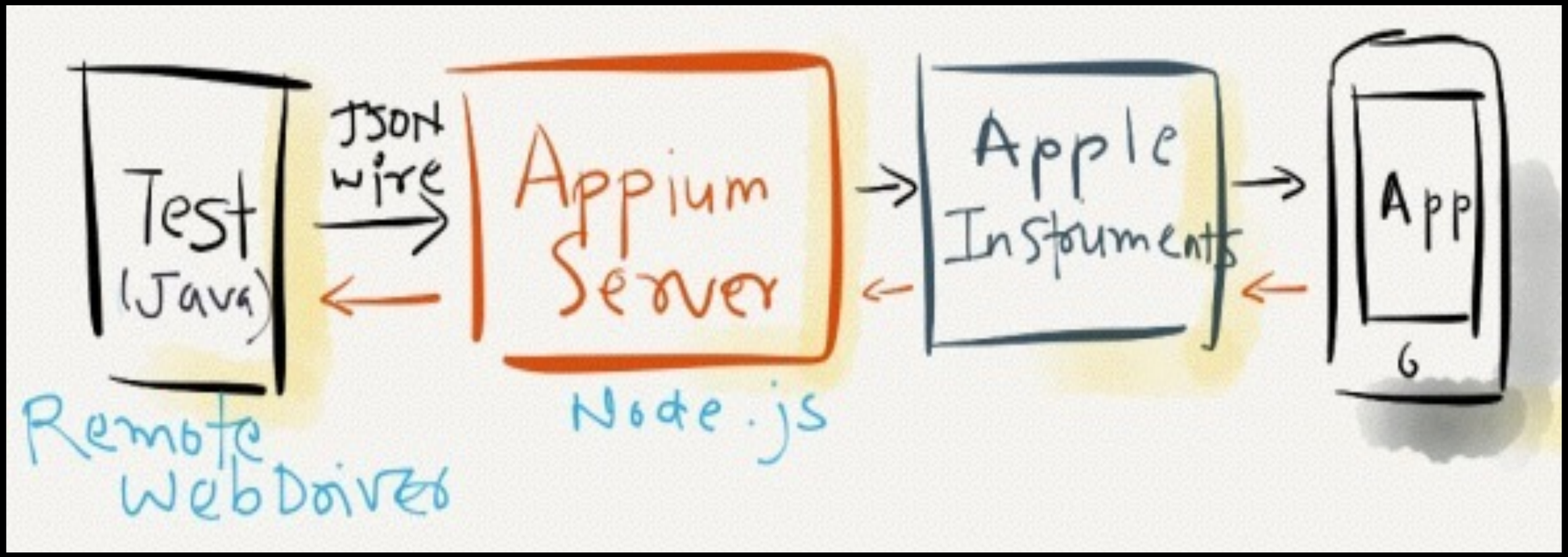
bootstrap.js

- 是一个UIAutomation 的脚本
- 是一个测试用例
- 这个测试用例是一个 while 循环
- while 循环里，使用 UIATarget.localTarget().host().performTaskWithPathArgumentsTimeout() 方法运行设备外部的 instruments-client.js
- instruments-client.js 连接前面我们说的instruments server，读出 command 返回给设备环境，并执行

iOS 脚本的执行过程

- elems = self.driver.find_elements_by_tag_name(‘textField')
- 进入appium server，找到对应的路由
- 由路由再映射到 controller 里的方法
- 在 controller 里面，根据device参数，具体到 iOS 的controller 里面的方法。
- iOS controller 的方法将 command 发给 instruments command server (Proxy)
- 一直运行着的 bootstrap，通过 instrument_client.js 监测到 command 进来，就拿过来执行，然后返回结果。

太复杂了...


Test
(Java)
JSON
wire
Appium
Server
Node.js
Apple
Instruments
App
Remote
WebDriver

Thank You